UNIT 3 DBMS

1.1 Table fundamentals

A **table** is database object that holds user data. The simplest analogy is to think of a table as a spread sheet. The cells of the spread sheet equate to the columns of a table having a specific **data type** associated with them. If the spread sheet cell has a number data type associated with it, then storing letters (i.e characters) in the same cell is not allowed. The same logic is applied to a table's column of the table will have specific data type bound to it. Oracle ensures that only data, which is identical to the data type or column, will be stored within the column.

1.2 Oracle data type

Each value manipulated by Oracle Database has a **data type**. The data type of a value associates a fixed set of properties with the value. These properties cause Oracle to treat values of one data type differently from values of another.

Data type	Description
CHAR(size)	A FIXED length string (can contain letters, numbers, and special characters). The *size* parameter specifies the column length in characters - can be from 0 to 255. Default is 1
VARCHAR(size)/ VARCHAR2(size)	A VARIABLE length string (can contain letters, numbers, and special characters). The maximum this data type can hold up to 4000 characters.
DATE	A date data type is used to represent date and time. Format: YYYY-MM-DD. The supported range is from January 1,4712BC to December 31, 4712AD
NUMBER(p,s)	Fixed precision and scale numbers. The p parameter indicates the maximum total number of digits that can be stored (both to the left and to the right of the decimal point). p must be a value from 1 to 38. Default is 18. The s parameter indicates the maximum number of digits stored to the right of the decimal point. s must be a value from 0 to p. Default value

	is 0
LONG	Stores variable length character strings containing upto 2 GB. LONG data can be used to store arrays of binary data in ASCII.
RAW/LONG RAW	It is used to store binary data, such as digitized picture or image. It can contain upto 4GB

1.3 CREATE TABLE command

The `CREATE TABLE` statement is used to create a new table in a database. For creating table we have to define the structure of table by adding name to columns, providing data type and size of data to be stored in columns. Each column definition is a single clause in the create table syntax. Each table column is separated from the other by a comma.

Syntax:

```
CREATE TABLE table_name (
    column1 datatype,
    column2 datatype,
    column3 datatype,
   ....
);
```

Example:

```
CREATE TABLE Persons (
    “PersonID” int,
    “Name” varchar(25),
    “Address” varchar(255),
    “City” varchar(255)
);
```

Rules for creating table

1. A name can have maximum upto 30 characters
2. Alphabets from A-Z, a-z and numbers are allowed
3. A name should begin with an alphabet
4. The use of the special character like _ is allowed and also recommend
5. SQL reserved words are not allowed. Example create, select etc

1.4 Inserting data into table

The `INSERT INTO` statement is used to insert new records in a table.

Syntax:

It is possible to write the `INSERT INTO` statement in two ways:

1. Specify both the column names and the values to be inserted to insert data into specified columns. :
   ```
   INSERT INTO table_name (column1, column2, column3, ...)
   VALUES (value1, value2, value3, ...);
   ```
2. 2. If you are adding values for all the columns of the table, you do not need to specify the column names in the SQL query. However, make sure the order of the values is in the same order as the columns in the table because table columns and value have one to one relation.
   ```
   INSERT INTO table_name
   VALUES (value1, value2, value3, ...);
   ```

Example:

```
INSERT INTO Customers (CustomerName, ContactName, Address, City,
PostalCode, Country)
VALUES ('Cardinal', 'Tom B. Erichsen', 'Skagen
21','Stavanger',4006,'Norway');
```

Here data should match the data type of the column.

1.5 Viewing data in the tables

The `SELECT` statement is used to select data from a database.

Filtering table data

The ways of filtering table data are :

- Selected columns and all rows

Syntax:

```
SELECT column1, column2, ... FROM table_name;
```

```
Example: SELECT CustomerName, City FROM Customers;
```

Here, column1, column2, ... are the field names of the table you want to select data from.

- All rows and all columns

If you want to select all the fields available in the table, use the following syntax:

```
SELECT * FROM table_name;
```

```
Example: SELEC * FROM Customers;
```

- Selected rows and all columns

  The WHERE clause is used with SELECT statement to return only those rows from the table, which satisfy the specified condition in the query.

  In SQL, the WHERE clause is not only used with SELECT, but it is also used with other SQL statements such as UPDATE, ALTER, and DELETE statements.

Syntax:

SELECT * **FROM** Name_of_Table **WHERE** [condition];

Example:

SELECT * **FROM** Student **WHERE** id=101;

- Selected rows and selected columns
  To view a specific set of rows and columns from a table.
  Syntax: **SELECT** *column1, column2, ...* **FROM** *table_name*
  WHERE [condition];
  Example:
  SELECT Student_Id, Age, Percentage, Grade **FROM** Employee **WHERE** id=101;

Eliminating Duplicate Rows using SELECT DISTINCT statement

Inside a table, a column often contains many duplicate values; and sometimes you only want to list the different (distinct) values.
The `SELECT DISTINCT` statement is used to return only distinct (different) values.

Syntax:

```
SELECT DISTINCT column1, column2, ...
FROM table_name;
```

```
Example :
```

```
SELECT DISTINCT Country FROM Customers;
```

1.6 Sorting data in a table

The ORDER BY statement in SQL is used to sort the fetched data in either ascending or descending according to one or more columns.

- By default ORDER BY sorts the data in ascending order.
- We can use the keyword DESC to sort the data in descending order and the keyword ASC to sort in ascending order.

Syntax:

```
SELECT * FROM table_name ORDER BY column_name ASC|DESC
```

ASC: to sort the data in ascending order.
DESC: to sort the data in descending order.

Example:

```
SELECT * FROM Student ORDER BY ROLL_NO DESC;
```

1.7 Creating a table from a table

Syntax:

A copy of an existing table can also be created using `CREATE TABLE`.

The new table gets the same column definitions. All columns or specific columns can be selected.

If you create a new table using an existing table, the new table will be filled with the existing values from the old table.

Syntax:

```
CREATE TABLE new_table_name (column1, column2,...) AS
    SELECT column1, column2,...
    FROM existing_table_name;
```

Example:

```
CREATE TABLE TestTable (name,contact) AS
SELECT customername, contactname
FROM customers;
```

To create a target table without the records from the source table, the select statement must have a WHERE clause. The WHERE clause must specify a condition that cannot be satisfied.

Syntax

```
CREATE TABLE new_table_name (column1, column2,...) AS
   SELECT column1, column2,...
   FROM existing_table_name WHERE ....;
```

Example:

```
CREATE TABLE new_table_name (column1, column2,...) AS
    SELECT column1, column2,...
    FROM existing_table_name WHERE 1=2;
```

1.8 Inserting data into a table from another table

The INSERT INTO SELECT statement copies data from one table and inserts it into another table.

The INSERT INTO SELECT statement requires that the data types in source and target tables match.

Syntax:

Copy all columns from one table to another table:

```
INSERT INTO table2 SELECT * FROM table1;
```

```
Example: INSERT INTO student SELECT * FROM sharada;
```

Inserting of a data set into a table from another table

```
INSERT INTO table2 SELECT column1, column2, column3, ...
FROM table1 WHERE condition;
```

```
Example:
```

```
INSERT INTO Customers SELECT SupplierName, City, Country FROM Suppliers
WHERE Country='Germany';
```

1.9 Delete Operation

The DELETE Statement in SQL is used to delete existing records from a table. We can delete a single record or all the records from the table.

- Removal of all the rows

Syntax: `DELETE FROM table_name;`

Example: `DELETE FROM customer;`

- Removal of specific rows

Syntax: `DELETE FROM table_name WHERE condition;`

Example: `DELETE FROM Customers WHERE CustomerName='Deepu';`

SQL Query to DELETE Certain Rows Based on data held by Other Tables

Since it is not possible to list more than one table in the FROM clause while performing a delete, the EXISTS clause can be used .

Example: Remove the address detail of the customer names Ivan

DELETE FROM ADDR_DTLS **WHERE EXISTS(SELECT** FNAME **FROM** CUST_MSTR **WHERE** CUST_MSTR.CUST_NO=ADDR_DTLS.CODE_NO **AND** CUST_MSTR.FNAME='Ivan');

1.10 Update the content of a table

The UPDATE statement is used to modify the existing records in a table.

- Updating all the rows of a table
  Syntax: `UPDATE table_nameSET column1 = value1, column2 = value2;`
  Example: `UPDATE Customers SET ContactName = 'SHARADA', City= 'DEVINAGARA'`
- Updating selected rows of a table
  Syntax: `UPDATE table_name SET column1 = value1, column2 = value2, ... WHERE condition;`

```
Example: UPDATE Customers
SET ContactName = 'Sharada', City= 'Talapady'
WHERE CustomerID = 1;
```

1.11 Modifying the structure of table

ALTER can be used to update the table's structure in the database

Adding new columns

SYNTAX :

```
ALTER TABLE table_name ADD (column_name datatype(size));
```

Example –

```
ALTER TABLE  Student ADD  (marks_obtained Number (3));
```

Alter table - drop column(To delete a column)

Syntax: **`ALTER TABLE table_name DROP COLUMN column_name;`**

Example: `ALTER TABLE Customers DROP COLUMN Email;`

Modifying the existing column

To change the data type of a column in a table, use the following syntax:

```
ALTER TABLE table_name Modify (column_name new_datatype(new_size));

ALTER TABLE Student Modify (name varchar(20));
```

Restriction on the ALTER TABLE

- We cannot change the name of the table
- We cannot change the name of the column
- We cannot decrease the size of the column if data exists

1.12 Renaming tables

The RENAME statement is used to change the table name.

Syntax: RENAME old_table _name To new_table_name ;

```
Example : RENAME College  To Sharada ;
```

1.13 Destroying Tables

The `DROP TABLE` statement is used to drop an existing table in a database

```
DROP TABLE table_name;
```

```
DROP TABLE Sharada;
```

1.14 Displaying table structure

To display information about the column defined in a table use the following syntax:

DESCRIBE table_name;

Example: DESCRIBE Sharada;

CHPATER 2 : DATA CONSTRAINTS

SQL constraints are used to specify rules for data in a table. Constraints can be specified when the table is created with the `CREATE TABLE` statement, or after the table is created with the `ALTER TABLE` statement.

Constraints are used to limit the type of data that can go into a table. This ensures the accuracy and reliability of the data in the table. If there is any violation between the constraint and the data action, the action is aborted. Constraints can be column level or table level. Column level constraints apply to a column, and table level constraints apply to the whole table.

2.1 Types of Data Constraints

There are two types, one is I/O constraints. It determines the speed at which data can be inserted or extracted from a table. The other type of constraints is called a business rule constraints.

2.1.1 I/O constraints

- **SQL PRIMARY KEY Constraint**

The `PRIMARY KEY` constraint uniquely identifies each record in a table. A table can have only ONE primary key; and in the table, this primary key can consist of single or multiple columns (fields).

Features of Primary Key

1. No duplicate values are allowed, i.e. Column assigned as primary key should have UNIQUE values only.
2. NO NULL values are present in column with Primary key. Hence there is Mandatory value in column having Primary key.
3. Only one primary key per table exist although Primary key may have multiple columns.
4. Defined in Create table / Alter table statement.
5. It is not compulsory, nut is recommended.
6. It cannot be LONG or LONG RAW datatype
7. It helps in relating tables with another
8. One table can combine upto 16 columns in composite primary key

SQL PRIMARY KEY at Column Level :

If Primary key contains just one column, it should be defined at column level.

Syntax: column_name datatype(size) PRIMARY KEY

Example :
```
CREATE TABLE Persons (
    ID int PRIMARY KEY,
    LastName varchar(255),
    FirstName varchar(255),
    Age int
);
```

SQL PRIMARY KEY at Table Level :

Whenever the primary key contains multiple columns it has to be specified at Table level.

Syntax: PRIMARY KEY(column_name,column_name)

Example:
```
CREATE TABLE Persons (
    ID int,
    LastName varchar(255),
    FirstName varchar(255),
```

```
    Age int,
    PRIMARY KEY (ID, LastName)
);
```

SQL FOREIGN KEY Constraint

- SQL `FOREIGN KEY` constraint is used to link two tables together and it is also termed as a referencing key.
- SQL `FOREIGN KEY` constraint is a field or collections of fields in one table that refers to the `PRIMARY KEY` constraint in another table.
- The table that contains `PRIMARY KEY` is termed as Master table or referenced table, and a table containing the `FOREIGN KEY` is termed as the Detail table.
- `FOREIGN KEY` constraint prevents invalid data being inserted into the `FOREIGN KEY` column because it has to be one of the values contained in the table it points to.

Features of FOREIGN KEY

- The parent used as a reference must be unique or primary key.
- Child may have duplicates as well as nulls.
- Parent records can be erased in the event that there is no child.
- Master table can't be updated in the event that a child exists.
- It is necessary to reference the primary key in the primary table.
- A foreign key column, as well as a constraint column must have the same data types.
- Records are not able to be added to a the child table if records in the master table doesn't exist.
- The master table's records can't be deleted even if corresponding records in the child table exits.

FOREIGN KEY at column level

Syntax: column_name datatype(size) REFERENCES table_name [(column_name)]
Example:
```
Create table people (no int references person(id),
                     Fname varchar2(20));
```

Here Person table should have primary key with type int. If there is single columnar Primary key in table, column name in syntax can be omitted.

SQL Foreign key at table level :

Syntax: FOREING KEY(column_name) **REFERENCES** table_name

(column_name)]

Example:

```
create table people(no varchar2(10),
                    fname varchar2(20),
                    foreign key(no) references person(id));
```

FOREIGN KEY Constraints defined with ON DELETE CASCADE

We can't delete parent table if its corresponding row exists in child table. Foreign keys with cascade delete means that if a parent table entry is deleted, the corresponding entries in the child table will be automatically deleted. This is called cascade deletion in Oracle.

For example:

```
CREATE TABLE supplier
( supplier_id numeric(10) >not null,
supplier_name varchar2(50) not null,
contact_name varchar2(50),
PRIMARY KEY (supplier_id)
);

CREATE TABLE products
( product_id numeric(10) not null,
supplier_id numeric(10) not null,
FOREIGN KEY (supplier_id)
REFERENCES supplier(supplier_id)
ON DELETE CASCADE
);
```

Now we can delete supplier table .

If you don't specify this option, then you'll get an error when you attempt to delete rows in the parent table(i.e supplier) , saying that "child rows exist".

FOREIGN KEY Constraints defined with ON DELETE SET NULL

A foreign key with " on delete set null " means that if a record in the parent table is deleted, then the corresponding records in the child table will have the foreign key fields set to NULL. The records in the child table will **not** be deleted in SQL Server.

```
CREATE TABLE products
( product_id INT PRIMARY KEY,
```

```
  product_name VARCHAR(50) NOT NULL,
  category VARCHAR(25)
);

CREATE TABLE inventory
( inventory_id INT PRIMARY KEY,
  product_id INT,
  quantity INT,
    FOREIGN KEY (product_id)
    REFERENCES products (product_id)
    ON DELETE SET NULL
);
```

Assigning user defined names to constraints

- In all databases whenever we are creating constraints database servers internally automatically creates unique identification number, for identifying constraint uniquely.
- Oracle also generates an unique identification number in the format of SYS_Cn for identifying constraint uniquely. This is called **predefined constraint name**.
- In place of this we can also create our own name by using constraint key word.This is called **user defined constraint name**.

Syntax : constraint constraint_name constrainttype

Example: **create table usertest(sno number (10) constraint pk primary key);**

SQL UNIQUE Constraint

- A unique key is a set of one or more than one fields/columns of a table that uniquely identify a record in a database table.
- You can say that it is little like primary key but it can accept only one null value and it cannot have duplicate values.
- The unique key and primary key both provide a guarantee for uniqueness for a column or a set of columns.
- There is an automatically defined unique key constraint within a primary key constraint.
- Unique key cannot be LONG or LONG RAW data type
- Unique key can combine upto 16 columns in a composite unique key

Unique key at column level

Syntax : <column_name><datatype>(size) UNIQUE

Example:

```
CREATE TABLE students
(
S_Id int  UNIQUE,
LastName varchar (255) ,
FirstName varchar (255),
City varchar (255)
)
```

Unique key at table level

Syntax: UNIQUE (column_name1,column_name2...)

Example:

```
   CREATE TABLE Persons (
   ID int NOT NULL,
   LastName varchar(255) NOT NULL,
   FirstName varchar(255),
   Age int,
   UNIQUE (ID)
);
```

2.2 Business Rule Constraints

A constraint business rule is used to manage multiple constraints on tables and columns in a database for the DBMS that support this feature.

During generation, the variable RULES is evaluated in order to generate a single constraint in the SQL script. This constraint is a concatenation of all check parameters defined in the Check tab of a table property sheet, and all validation business rules applied to the current table.

Business rule validation checks are performed when any table write operation is carried out. Any insert or update statement causes the relevant Check constraints to be evaluated. The rules must be satisfied while inserting data into table

Column level constraints

If data constraints are defined as an attribute of a column definition when creating or altering a table structure, they are column level constraints.

Table level constraints

If data constraints are defined after defining all the tables column attributes when creating or altering a table structure, it is a table level constraints

NULL Value concept

field with a NULL value is a field with no value.

If a field in a table is optional, it is possible to insert a new record or update a record without adding a value to this field. Then, the field will be saved with a NULL value.

Principles of NULL values:

- Setting a NULL value is appropriate when the actual value is unknown, or when a value would not be meaningful.
- A NULL value is not equivalent to a value of ZERO if the data type is a number and is not equivalent to spaces if the data type is character.
- A NULL value can be inserted into columns of any data type.
- A NULL value will evaluate NULL in any expression.
- Suppose if any column has a NULL value, then UNIQUE, FOREIGN key, CHECK constraints will ignore by SQL.

Difference between an empty string and a null value

Now an empty string is treated as a null value in Oracle. Let's demonstrate.

We've created a table called suppliers with the following table definition:

```
create table suppliers
( supplier_id number,
  supplier_name varchar2(100)
);
```

Next, we'll insert two records into this table.

```
insert into suppliers (supplier_id, supplier_name )
values ( 10565, null );


insert into suppliers (supplier_id, supplier_name )
values ( 10567, '' );
```

The first statement inserts a record with a *supplier_name* that is null, while the second statement inserts a record with an empty string as a *supplier_name*.

Now, let's retrieve all rows with a supplier_name that is an empty string value as follows:

```
select * from suppliers
where supplier_name = '';
```

When you run this statement, you'd expect to retrieve the row that you inserted above. But instead, this statement will not retrieve any records at all.

Now, try retrieving all records where the supplier_name contains a null value:

```
select * from suppliers
where supplier_name is null;
```

When you run this statement, you will retrieve both rows. This is because Oracle has now changed its rules so that empty strings behave as null values.

With respect to using null values in SQL statements in Oracle, it is also important to note that the null value is unique in that you can not use the usual operands (=, <, >, etc) on a null value. Instead, you must use the IS NULL and IS NOT NULL conditions.

SQL NOT NULL Constraint

By default, a column can hold NULL values. The `NOT NULL` constraint enforces a column to NOT accept NULL values. This enforces a field to always contain a value, which means that you cannot insert a new record, or update a record without adding a value to this field.

SQL NOT NULL on CREATE TABLE

The following SQL ensures that the "ID", "LastName", and "FirstName" columns will NOT accept NULL values when the "Persons" table is created:

Example

```
CREATE TABLE Persons (
    ID int NOT NULL,
    LastName varchar(255) NOT NULL,
    FirstName varchar(255) NOT NULL,
    Age int
);
```

Check constraint

The CHECK constraint is used to limit the value range that can be placed in a column.

If you define a CHECK constraint on a column it will allow only certain values for this column.

If you define a CHECK constraint on a table it can limit the values in certain columns based on values in other columns in the row.

Syntax:

CREATE TABLE TableName(ColumnName1 Datatype(**size**), ColumnName2 Datatype(**size**), ColumnName3 Datatype(**size**) **Constraint** ConstraintName **CHECK**(ColumnName CON DITION Value),…, ColumnNameN Datatype(**size**));

Example :

The following SQL creates a CHECK constraint on the "Age" column when the "Persons" table is created. The CHECK constraint ensures that the age of a person must be 18, or older:

```
CREATE TABLE Persons (
    ID int NOT NULL,
    LastName varchar(255) NOT NULL,
    FirstName varchar(255),
    Age int,
    CHECK (Age>=18)
);
```

To allow naming of a CHECK constraint, and for defining a CHECK constraint on multiple columns, use the following SQL syntax:

```
CREATE TABLE Persons (
    ID int NOT NULL,
    LastName varchar(255) NOT NULL,
    FirstName varchar(255),
    Age int,
    City varchar(255),
    CONSTRAINT CHK_Person CHECK (Age>=18 AND City='Sandnes')
);
```

```
CREATE TABLE pets(
        ID INT NOT NULL,
```

```
Name VARCHAR(30) NOT NULL,
Age INT,
GENDER VARCHAR(9),
PRIMARY KEY(ID),
check(GENDER in ('Male', 'Female'))
);
```

Here gender column accepts only ‘male’ and ‘female’ input. If you tries to enter other than these input , then it will give error.

SQL DEFAULT Constraint

The `DEFAULT` constraint is used to set a default value for a column.

The default value will be added to all new records, if no other value is specified.

The following SQL sets a `DEFAULT` value for the "India" column when the "Persons" table is created:

```
CREATE TABLE Persons (
    ID int NOT NULL,
    LastName varchar(255) NOT NULL,
    FirstName varchar(255),
    Age int,
    City varchar(255) DEFAULT 'India'
);
```

Chapter 3: Computation done on table data

Arithmetic Operators

In Structured Query Language, the arithmetic operators are used to perform mathematical operations on the numerical values stored in the database tables.

We can use these operators with the SELECT statement in SQL. We can also use the WHERE clause in the SELECT statement for performing operations on particular rows.

These types of operators are used between two numerical operands for performing addition, subtraction, multiplication, and division operations.

The arithmetic operators in SQL are categorized into the following five types:

1. SQL Addition Operator (+)
2. SQL Subtraction Operator (-)
3. SQL Multiplication Operator (*)
4. SQL Division Operator (/)
5. SQL Modulus Operator (%)

Syntax:

```
SELECT <Expression>[arithmetic operator]<expression>...
 FROM [table_name]
 WHERE [expression];
```

1. Addition Operator (+)

The operator '+' is used to perform addition operation on two operands.
Example **SELECT** Emp_Salary + Emp_Bonus **AS** Emp_Total_Salary **FROM** Employee;

Output:

Emp_Total_Salary
27000
30000
37500
25000

SQL Subtraction Operator (-)

The SQL Subtraction Operator performs the subtraction on the numerical columns in the table.

If we want to subtract the values of one numerical column from the values of another numerical column, then we have to specify both columns as the first and second operand. We can also subtract the integer value from the values of the integer column.

Example:

```
SELECT Emp_Salary - Emp_Panelty AS Emp_Total_Salary FROM Employee;
```

SQL Multiplication Operator (*)

The SQL Multiplication Operator performs the multiplication on the numerical columns in the table.

If you want to multiply the values of two numerical columns, then you have to specify both columns as the first and second operand. You can also multiply the integer value with the values of an integer column.

1. **SELECT** Car_Amount * Car_Price **AS** Car_Total_Price **FROM** Cars;

SQL Division Operator (/)

The SQL Division operator divides the numerical values of one column by the numerical values of another column.

```
SELECT Car_Price / Car_Amount AS One_Car_Price FROM Cars;
```

SQL Modulus Operator (%)

The SQL Modulus Operator provides the remainder when the numerical values of one column are divided by the numerical values of another column.

Example: **SELECT** Student_English % Student_Maths **AS** Remainder **FROM** Student;

SQL LOGICAL OPERATORS

The Logical Operator is nothing but which returns the result in one form, i.e., either it will display the query is true, or the query is false. The results displayed to combine or merge more than one true or false data.

The Logical Operators in SQL are as follows:

1. SQL AND OPERATOR
2. SQL OR OPERATOR
3. SQL NOT OPERATOR
4. SQL BETWEEN OPERATOR
5. SQL IN OPERATOR
6. SQL LIKE OPERATOR

1. SQL AND Operator

The SQL AND operator is used with the where clause in the SQL Query. AND operator in SQL returns only those records which satisfy both the conditions in the SQL query.

Example: **Write a query to retrieve only those records of employees from the employees table where the designation is 'Project Manager' and the City to which the employee belongs to is Mumbai.**

Query: `SELECT * FROM employees WHERE City = "Mumbai" AND Designation = "Project Manager";`

2.SQL OR Operator

The SQL OR operator is used with the where clause in an SQL Query. AND operator in SQL returns only those records that satisfy any of the conditions in the SQL query.

Example:

Write a query to retrieve only those records of employees from the employees table where the employee's designation is 'System Engineer' or the city to which the employee belongs is Mumbai.

Query: `SELECT * FROM employees WHERE Designation = "System Engineer" OR City = "Mumbai";`

3. SQL NOT Operator

NOT operator in SQL shows those records from the table where the criteria is not met. NOT operator is used with where clause in a SELECT query.

Let's understand the below example, which explains how to execute NOT operator in SQL query.

Example:

Write a query to retrieve only those records of employees from the employees table where the employee's designation is not Project Manager.

Query: **SELECT** * **FROM** employees **WHERE** NOT Designation = "Project Manager";

Range searching

1.SQL BETWEEN Operator

This operator displays the records which fall between the given ranges in the SQL query. The results of the BETWEEN operator include begin and end values of the given range.

Example:

Write a query to retrieve only those records of an employee from the employees table where employee salary lies between 50000 to 90000.

Query: **SELECT** * **FROM** employees **WHERE** Salary BETWEEN 50000 AND 90000;

Pattern matching

1. SQL LIKE Operator

LIKE Operator in SQL displays only those data from the table which matches the pattern specified in the query. Percentage (%) and underscore (_) are the two wildcard operators used with LIKE Operator to perform pattern matching tasks.

- Let's understand the below example, which explains how to execute the LIKE operator in an SQL query.

Example: **Write a query to retrieve only those records of employees from the employees table whose name starts with the letter S.**

Query: **SELECT** * **FROM** employees **WHERE** Salary LIKE "S%";

- The underscore character **(_)** represents a single character to match a pattern from a word or string. More than one (_) underscore characters can be used to match a pattern of multiple characters.

Query: **SELECT** * **FROM** employees **WHERE** id LIKE "S_";

Here output display id such as S1, S2, SQ, Sr.... etc

2. SQL IN Operator

When we want to check for one or more than one value in a single SQL query, we use IN operator with the WHERE clause in a SELECT query.

Let's understand the below example, which explains how to execute IN operator in an SQL query.

Example:

Write a query to retrieve only those records of employees from the employees table where the city to which the employee belongs to is either Mumbai, Bangalore, or Pune.

Query:

```
SELECT * FROM employees WHERE City IN ("Mumbai", "Bangalore", "Pune");
```

Oracle Table Dual

Oracle DUAL table which is a special table used for evaluating expressions or calling functions.

In Oracle, the SELECT statement must have a table name in FROM clause. Otherwise the SELECT fails.

When an arithmetic exercise is to be performed such as 2*2 or 4/2 and so on, there is no table being referenced, only numeric literals are being used.

To facilitate such calculation via a SELECT , Oracle provides a **dummy table** called **DUAL**.

Structure of DUAL table is

```
SQL> desc dual;

 Name                          Null?    Type

 ----------------------------------------- -------- ----------------------------

 DUMMY                                    VARCHAR2(1)
```

Content of DUAL

```
SQL> select * from dual ;
```

D

X

Example : SELECT 2*2 FROM DUAL;

OUTPUT

2*2

4

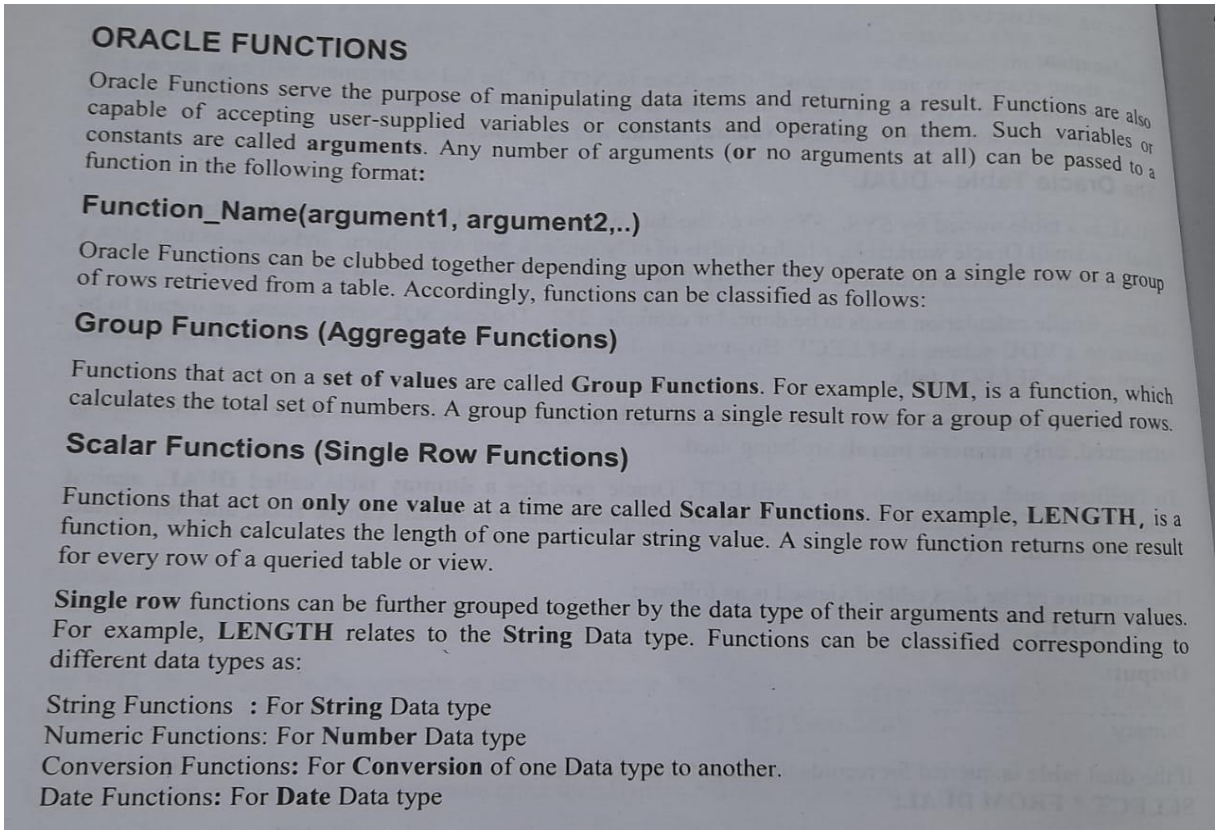
Example:

SQL> select sysdate from dual;

SYSDATE

22-NOV-22

Oracle Functions:



ORACLE FUNCTIONS

Oracle Functions serve the purpose of manipulating data items and returning a result. Functions are also capable of accepting user-supplied variables or constants and operating on them. Such variables or constants are called **arguments**. Any number of arguments (**or** no arguments at all) can be passed to a function in the following format:

Function_Name(argument1, argument2,..)

Oracle Functions can be clubbed together depending upon whether they operate on a single row or a group of rows retrieved from a table. Accordingly, functions can be classified as follows:

Group Functions (Aggregate Functions)

Functions that act on a **set of values** are called **Group Functions**. For example, **SUM**, is a function, which calculates the total set of numbers. A group function returns a single result row for a group of queried rows.

Scalar Functions (Single Row Functions)

Functions that act on **only one value** at a time are called **Scalar Functions**. For example, **LENGTH**, is a function, which calculates the length of one particular string value. A single row function returns one result for every row of a queried table or view.

Single row functions can be further grouped together by the data type of their arguments and return values. For example, **LENGTH** relates to the **String** Data type. Functions can be classified corresponding to different data types as:

String Functions : For **String** Data type
Numeric Functions: For **Number** Data type
Conversion Functions: For **Conversion** of one Data type to another.
Date Functions: For **Date** Data type

Aggregate function

AVG function

The AVG function is used to calculate the average value of the numeric type. AVG function returns the average of all non-Null values.

Syntax

AVG([ALL|DISTINCT] expression)

Example:

1. SELECT AVG(COST)"Average" FROM PRODUCT_MAST;

MIN Function

MIN function is used to find the minimum value of a certain column. This function determines the smallest value of all selected values of a column.

Syntax

MIN([ALL|DISTINCT] expression)

Example:

1. SELECT MIN(RATE) FROM PRODUCT_MAST;

MAX Function

MAX function is used to find the maximum value of a certain column. This function determines the largest value of all selected values of a column.

MAX([ALL|DISTINCT] expression)

Example:

1. SELECT MAX(RATE) FROM PRODUCT_MAST;

COUNT FUNCTION

- COUNT function is used to Count the number of rows in a database table. It can work on both numeric and non-numeric data types.

- COUNT function uses the COUNT(*) that returns the count of all the rows in a specified table. COUNT(*) considers duplicate and Null.

Syntax: COUNT(*)
or
COUNT([ALL|DISTINCT] expression)

Example:

1. SELECT COUNT(*) “Total Number of Rows” FROM PRODUCT_MAST;
2. SELECT COUNT(*) FROM PRODUCT_MAST WHERE RATE>=20;
3. SELECT COUNT(DISTINCT COMPANY) FROM PRODUCT_MAST;

SUM Function

Sum function is used to calculate the sum of all selected columns. It works on numeric fields only.

Syntax

SUM()
or
SUM([ALL|DISTINCT] expression)

Example: SUM()

1. SELECT SUM(COST) FROM PRODUCT_MAST;

Grouping the data from the table

GROUPBY clause

The SQL GROUP BY clause can be used in a SELECT statement to collect data across multiple records and group the results by one or more columns.

Syntax

The syntax for the GROUP BY clause in SQL is:

```
SELECT columnName1, columnName2, ... columnnName_n,
```

```
       aggregate_function (aggregate_expression)
FROM tables
[WHERE conditions]
GROUP BY columnName1, columnName2, ... columnmnName_n;
```

Example - Using GROUP BY with the SUM Function

Let's look at how to use the GROUP BY clause with the SUM function in SQL.

In this example, we have a table called *employees* with the following data:

employee_number	last_name	first_name	salary	dept_id
1001	Smith	John	62000	500
1002	Anderson	Jane	57500	500
1003	Everest	Brad	71000	501
1004	Horvath	Jack	42000	501

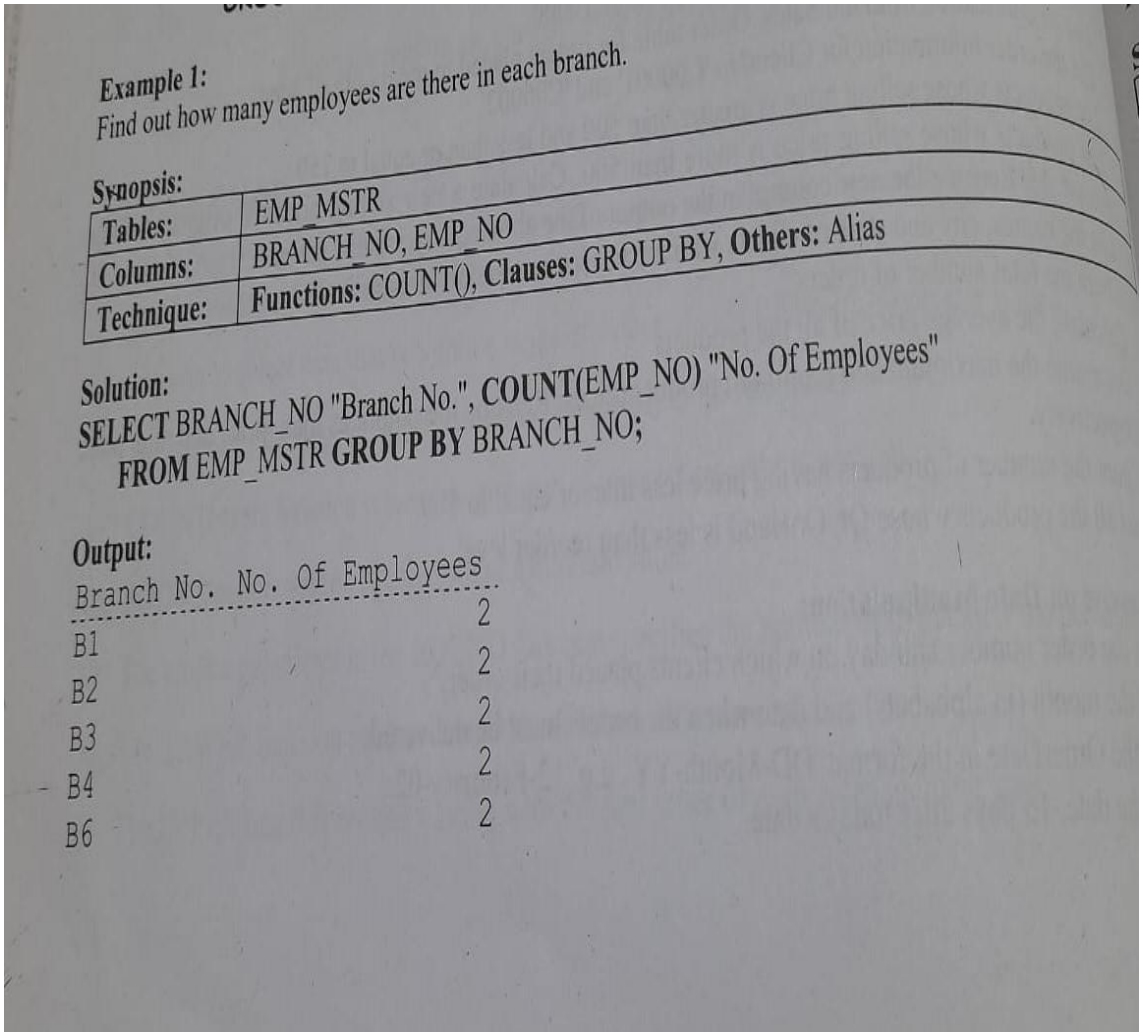
Enter the following SQL statement:

```
SELECT dept_id, SUM(salary) AS total_salaries
FROM employees
GROUP BY dept_id;
```

There will be 2 records selected. These are the results that you should see:

dept_id	total_salaries
500	119500
501	113000

In this example, we've used the SUM function to add up all of the salaries for each *dept_id* and we've aliased the results of the SUM function as *total_salaries*. Because the *dept_id* is not encapsulated in the SUM function, it must be listed in the GROUP BY clause.



Example 1:
Find out how many employees are there in each branch.

Synopsis:

Tables:	EMP_MSTR
Columns:	BRANCH_NO, EMP_NO
Technique:	**Functions:** COUNT(), **Clauses:** GROUP BY, **Others:** Alias

Solution:

```
SELECT BRANCH_NO "Branch No.", COUNT(EMP_NO) "No. Of Employees"
   FROM EMP_MSTR GROUP BY BRANCH_NO;
```

Output:

Branch No.	No. Of Employees
B1	2
B2	2
B3	2
B4	2
B6	2

Explanation:

In the above example, the data that has to be retrieved is available in the **EMP_MSTR** table. Since the **number of employees** per branch is required, the records need to be **grouped** on the basis of field **BRANCH_NO** and then the **COUNT()** function must be applied to the field **EMP_NO** which calculates the number of employees on a per branch basis.

Example 2:

Find out the total number of (Current and Savings Bank) accounts verified by each employee.

Synopsis:

Tables:	ACCT_MSTR
Columns:	VERI_EMP_NO, ACCT_NO
Technique:	**Functions:** COUNT(), **Clauses:** GROUP BY, **Others:** Alias

Solution:

```
SELECT VERI_EMP_NO "Emp. No.", COUNT(ACCT_NO) "No. Of A/Cs Verified"
   FROM ACCT_MSTR GROUP BY VERI_EMP_NO;
```

Output:

```
Emp. No.  No. Of A/Cs Verified
E1                           7
E4                           8
```

Explanation:

In the above example, the data that has to be retrieved is available in the **ACCT_MSTR** table. Since the **number of accounts verified** per employee is required, the records need to be **grouped** on the basis of field **VERI_EMP_NO** and then the **COUNT()** function is applied to the field **ACCT_NO** which calculates the number of accounts verified per employee.

Example 3:

Find out the total number of accounts segregated on the basis of account type per branch.

Synopsis:

Tables:	ACCT_MSTR
Columns:	BRANCH_NO, TYPE, ACCT_NO
Technique:	**Functions:** COUNT(), **Clauses:** GROUP BY, **Others:** Alias

Solution:

```
SELECT BRANCH_NO "Branch No.", TYPE "A/C Type", COUNT(ACCT_NO) "No. Of A/Cs"
   FROM ACCT_MSTR GROUP BY BRANCH_NO, TYPE;
```

Output:

```
Branch No.  A/C Type  No. Of A/Cs
B1          CA                  1
B1          SB                  2
B2          CA                  2
B2          SB                  1
B3          SB                  2
B4          SB                  2
B5          CA                  2
B6          CA                  1
B6          SB                  2
9 rows selected.
```

SQL: HAVING Clause

The SQL HAVING clause is used in combination with the GROUP BY clause to restrict the groups of returned rows to only those whose the condition is TRUE.

Syntax

The syntax for the HAVING clause in SQL is:

```
SELECT columnName1, columnName2, ... columnnName_n,
       aggregate_function (aggregate_expression)
FROM tables
[WHERE conditions]
GROUP BY columnName1, columnName2, ... columnnName_n,
HAVING condition;
```

Example - Using SUM function

Let's look at a SQL HAVING clause example that uses the SQL SUM function.

You could also use the SQL SUM function to return the name of the department and the total sales (in the associated department). The SQL HAVING clause will filter the results so that only departments with sales greater than $1000 will be returned.

```
SELECT department, SUM(sales) AS "Total sales"
FROM order_details
GROUP BY department
HAVING SUM(sales) > 1000;
```

Example - Using COUNT function

Let's look at how we could use the HAVING clause with the SQL COUNT function.

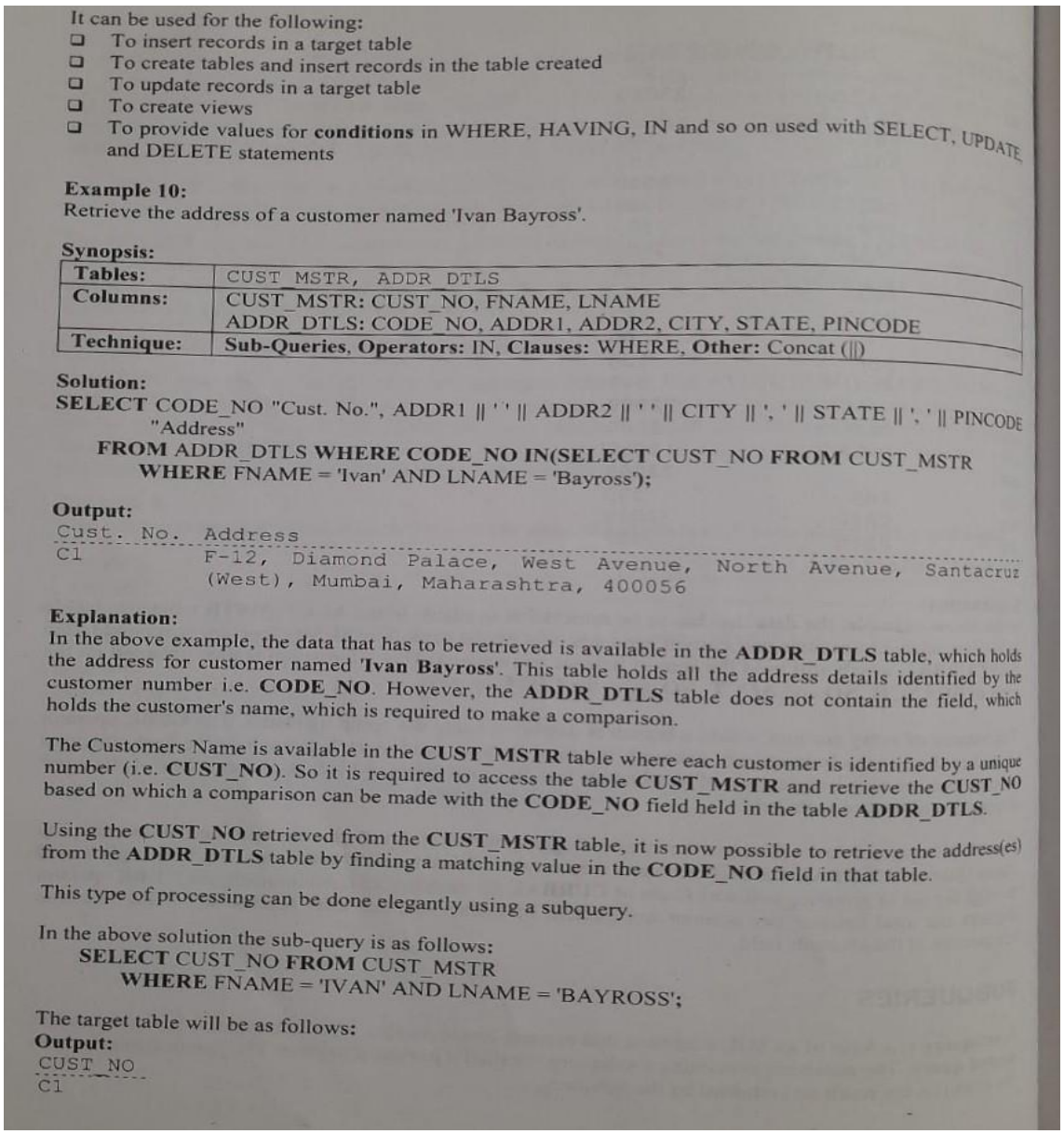
You could use the SQL COUNT function to return the name of the department and the number of employees (in the associated department) that make over $25,000 / year. The SQL HAVING clause will filter the results so that only departments with more than 10 employees will be returned.

```
SELECT department, COUNT(*) AS "Number of employees"
```

```
FROM employees
WHERE salary > 25000
GROUP BY department
HAVING COUNT(*) > 10;
```

SUBQUERIES

A subquery is a form of an SQL statement that appears inside another SQL statement. It also termed as nested query. The statement containing a subquery is called a parent statement. The parent statement uses the rows(i.e the result set) returned by subquery.



It can be used for the following:

- To insert records in a target table
- To create tables and insert records in the table created
- To update records in a target table
- To create views
- To provide values for **conditions** in WHERE, HAVING, IN and so on used with SELECT, UPDATE and DELETE statements

Example 10:
Retrieve the address of a customer named 'Ivan Bayross'.

Synopsis:

Tables:	CUST_MSTR, ADDR_DTLS
Columns:	CUST_MSTR: CUST_NO, FNAME, LNAME ADDR_DTLS: CODE_NO, ADDR1, ADDR2, CITY, STATE, PINCODE
Technique:	**Sub-Queries, Operators:** IN, **Clauses:** WHERE, **Other:** Concat (||)

Solution:

```
SELECT CODE_NO "Cust. No.", ADDR1 || ' ' || ADDR2 || ' ' || CITY || ', ' || STATE || ', ' || PINCODE
       "Address"
   FROM ADDR_DTLS WHERE CODE_NO IN(SELECT CUST_NO FROM CUST_MSTR
      WHERE FNAME = 'Ivan' AND LNAME = 'Bayross');
```

Output:

```
Cust. No.  Address
C1         F-12, Diamond Palace, West Avenue, North Avenue, Santacruz
           (West), Mumbai, Maharashtra, 400056
```

Explanation:
In the above example, the data that has to be retrieved is available in the **ADDR_DTLS** table, which holds the address for customer named **'Ivan Bayross'**. This table holds all the address details identified by the customer number i.e. **CODE_NO**. However, the **ADDR_DTLS** table does not contain the field, which holds the customer's name, which is required to make a comparison.

The Customers Name is available in the **CUST_MSTR** table where each customer is identified by a unique number (i.e. **CUST_NO**). So it is required to access the table **CUST_MSTR** and retrieve the **CUST_NO** based on which a comparison can be made with the **CODE_NO** field held in the table **ADDR_DTLS**.

Using the **CUST_NO** retrieved from the **CUST_MSTR** table, it is now possible to retrieve the address(es) from the **ADDR_DTLS** table by finding a matching value in the **CODE_NO** field in that table.

This type of processing can be done elegantly using a subquery.

In the above solution the sub-query is as follows:

```
SELECT CUST_NO FROM CUST_MSTR
   WHERE FNAME = 'IVAN' AND LNAME = 'BAYROSS';
```

The target table will be as follows:

Output:

```
CUST_NO
C1
```

The outer sub-query output will simplify the solution as shown below:

```
SELECT CODE_NO "Cust. No.", ADDR1 || ' ' || ADDR2 || ' ' || CITY || ', ' || STATE || ', ' || PINCODE
       "Address" FROM ADDR_DTLS WHERE CODE_NO IN('C1');
```

When the above SQL query is executed the resulting output is equivalent to the desired output of the SQL query using two levels of Sub-queries.

Example 11:

Find the customers who do not have bank branches in their vicinity.

Synopsis:

Tables:	CUST_MSTR, ADDR_DTLS
Columns:	CUST_MSTR: CUST_NO, FNAME, LNAME ADDR_DTLS: CODE_NO, PINCODE
Technique:	**Sub-Queries, Operators:** IN, **Clauses:** WHERE, **Other:** Concat (||)

Solution:

```
SELECT (FNAME || ' ' || LNAME) "Customer" FROM CUST_MSTR
   WHERE CUST_NO IN(SELECT CODE_NO FROM ADDR_DTLS
      WHERE CODE_NO LIKE 'C%' AND PINCODE NOT IN(SELECT PINCODE
      FROM ADDR_DTLS WHERE CODE_NO LIKE 'B%'));
```

Output:

```
Customer
------------------------
Ivan Bayross
Namita Kanade
Chriselle Bayross
Mamta Muzumdar
Chhaya Bankar
Ashwini Joshi
Hansel Colaco
Anil Dhone
Alex Fernandes
Ashwini Apte
10 rows selected.
```

Explanation:

In the above example, the data that has to be retrieved is available in the **CUST_MSTR**, which holds the Customer details. The **CUST_MSTR** table will only provide all the customer names but to retrieve only those customers who do not have any bank branches in their vicinity, one more tables will be involved that is **ADDR_DTLS** table, which holds the branch as well as customer addresses. This table holds all the address details identified by the branch / customer number i.e. **CODE_NO**. This table will help comparing the value held in the **PINCODE** field belonging to the customer addresses with the ones of bank branch addresses.

To understand the solution the query mentioned above needs to be simplified. The inner most sub-queries should be handled first and then proceeded outwards.

The **first step** is to identify the vicinities in which the branches are located. This is done by extracting the **PINCODE** from the address details table (i.e. **ADDR_DTLS**) for all entries belonging to the branches. The SQL query for this will be as follows:

```
SELECT PINCODE FROM ADDR_DTLS WHERE CODE_NO LIKE 'B%'
```

The target table will be as follows:

Output:

```
PINCODE
------------
400057
400058
400004
400045
400078
110004
6 rows selected.
```

The resulting output simplifies the solution as shown below:

```
SELECT (FNAME || ' ' || LNAME) "Customer" FROM CUST_MSTR
    WHERE CUST_NO IN(SELECT CODE_NO FROM ADDR_DTLS
        WHERE CODE_NO LIKE 'C%' AND PINCODE NOT IN('400057', '400058',
                                                   '400004', '400045', '400078', '110004'));
```

The **second step** is to identify the customers who are not resident near a branch. To do this the customer numbers (i.e. **CODE_NO**) have to be retrieved from the Address details table (i.e. **ADDR_DTLS**). The SQL query for this will be as follows:

```
SELECT CODE_NO FROM ADDR_DTLS WHERE CODE_NO LIKE 'C%'
    AND PINCODE NOT IN('400057', '400058', '400004', '400045', '400078', '110004')
```

The target table will be as follows:

Output:

```
CODE_NO
-------
C1
C2
C3
C4
C5
C6
C7
C8
C9
C10
10 rows selected.
```

The outer sub-query output will simplify the solution as shown below:

```
SELECT (FName || ' ' || LName) "Customer" FROM Cust_Mstr
    WHERE Cust_No IN('C1', 'C2', 'C3', 'C4', 'C5', 'C6', 'C7', 'C8', 'C9', 'C10');
```

When the above SQL query is executed the resulting output is equivalent to the desired output of the SQL query using two levels of Sub-queries.

Example 12:

List customers holding Fixed Deposits in the bank of amount more than 5,000.

Synopsis:

Tables:	CUST_MSTR, ACCT_FD_CUST_DTLS, FD_DTLS
Columns:	CUST_MSTR: CUST_NO, FNAME, LNAME ACCT_FD_CUST_DTLS: CUST_NO, ACCT_FD_NO FD_DTLS: FD_SER_NO, AMT
Technique:	**Sub-Queries, Operators:** IN() **Clauses:** WHERE, **Other:** Concat (||)

Solution:

```
SELECT (FNAME || ' ' || LNAME) "Customer" FROM CUST_MSTR WHERE CUST_NO
    IN(SELECT CUST_NO FROM ACCT_FD_CUST_DTLS WHERE ACCT_FD_NO
        IN(SELECT FD_SER_NO FROM FD_DTLS WHERE AMT > 5000));
```

Output:

```
Customer
-------------------------
Chriselle Bayross
Mamta Muzumdar
Chhaya Bankar
Áshwini Joshi
```

Explanation:

In the above example, the data that has to be retrieved is available in the **CUST_MSTR**, which holds the Customer details. The **CUST_MSTR** table will only provide all the customer names but to retrieve only those customers who hold fixed deposits of amount exceeding Rs.5000, two more tables will be involved that is **ACCT_FD_DTLS** and **FD_DTLS.** The **ACCT_FD_DTLS** acts as a link between the **CUST_MSTR** and **FD_DTLS** table and hold details identified by the **ACCT_FD_NO** field. The **FD_DTLS** table actually hold all the details related to the fixed deposits held by the customers in a bank identified by the **FD_SER_NO** field.

To understand the solution the query mentioned above needs to be simplified. The inner most sub-queries should be handled first and then proceeded outwards.

The **first step** is to identify the fixed deposits of amount more than 5000. This is done by extracting the value held by the **FD_SER_NO** field from the fixed deposits details table (i.e. **FD_DTLS**). The SQL query for this will be as follows:

```
SELECT FD_SER_NO FROM FD_DTLS WHERE AMT > 5000
```

The target table will be as follows:

Output:

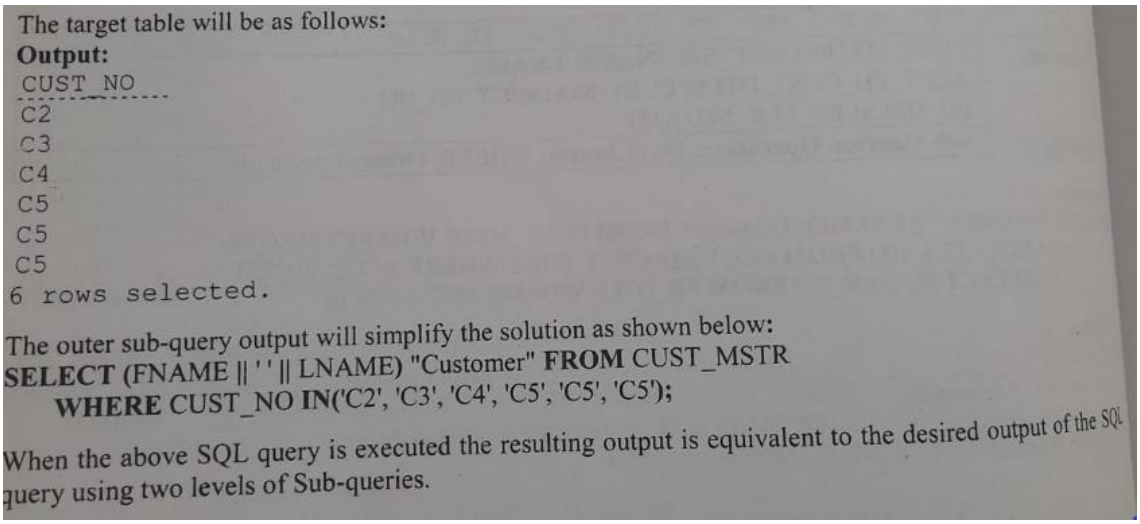
```
FD_SER_NO
---------
FS1
FS2
FS2
FS5
```

The data retrieved by the above **SELECT** statement will be passed to the outer sub-query as in:

```
SELECT (FNAME || ' ' || LNAME) "Customer" FROM CUST_MSTR WHERE CUST_NO
    IN(SELECT CUST_NO FROM ACCT_FD_CUST_DTLS WHERE ACCT_FD_NO
        IN('FS1', 'FS2', 'FS2', 'FS5'));
```

The **second step** is to identify the customer numbers who hold these FDs i.e. ('FS1', 'FS2', 'FS2', 'FS5'). To do this the customer numbers (i.e. **CUST_NO**) have to be retrieved from the Account FD details table (i.e. **ACCT_FD_DTLS**). The SQL query for this will be as follows:

```
SELECT CUST_NO FROM ACCT_FD_CUST_DTLS
    WHERE ACCT_FD_NO IN('FS1', 'FS2', 'FS2', 'FS5')
```



The target table will be as follows:

Output:

```
CUST_NO
-------
C2
C3
C4
C5
C5
C5
6 rows selected.
```

The outer sub-query output will simplify the solution as shown below:

```
SELECT (FNAME || ' ' || LNAME) "Customer" FROM CUST_MSTR
    WHERE CUST_NO IN('C2', 'C3', 'C4', 'C5', 'C5', 'C5');
```

When the above SQL query is executed the resulting output is equivalent to the desired output of the SQL query using two levels of Sub-queries.

SQL: JOINS

SQL JOINS are used to retrieve data from multiple tables. A SQL JOIN is performed whenever two or more tables are listed in a SQL statement.

There are 4 different types of SQL joins:

- SQL INNER JOIN (sometimes called simple join)
- SQL LEFT OUTER JOIN (sometimes called LEFT JOIN)
- SQL RIGHT OUTER JOIN (sometimes called RIGHT JOIN)
- SQL FULL OUTER JOIN (sometimes called FULL JOIN)
- CROSS JOIN

SQL INNER JOIN (simple join)

Chances are, you've already written a SQL statement that uses an SQL INNER JOIN. It is the most common type of SQL join. SQL INNER JOINS return all rows from multiple tables where the join condition is met.

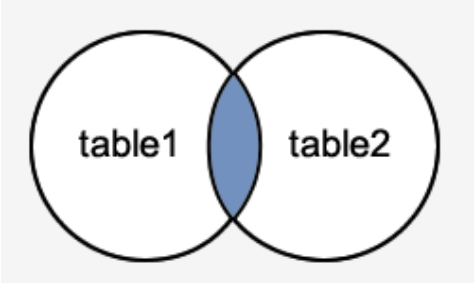
Syntax

The syntax for the INNER JOIN in SQL is:

```
SELECT columns
FROM table1
INNER JOIN table2
ON table1.column = table2.column;
```

Visual Illustration

In this visual diagram, the SQL INNER JOIN returns the shaded area:



The SQL INNER JOIN would return the records where *table1* and *table2* intersect.

Example

Let's look at an example of how to use the INNER JOIN in a query.

In this example, we have a table called *customers* with the following data:

customer_id	last_name	first_name	favorite_website
4000	Jackson	Joe	techonthenet.com
5000	Smith	Jane	digminecraft.com
6000	Ferguson	Samantha	bigactivities.com
7000	Reynolds	Allen	checkyourmath.com
8000	Anderson	Paige	NULL
9000	Johnson	Derek	techonthenet.com

And a table called *orders* with the following data:

order_id	customer_id	order_date
1	7000	2016/04/18
2	5000	2016/04/18

order_id	customer_id	order_date
3	8000	2016/04/19
4	4000	2016/04/20
5	NULL	2016/05/01

Enter the following SQL statement:

```
SELECT customers.customer_id, orders.order_id, orders.order_date
FROM customers
INNER JOIN orders
ON customers.customer_id = orders.customer_id
ORDER BY customers.customer_id;
```

There will be 4 records selected. These are the results that you should see:

customer_id	order_id	order_date
4000	4	2016/04/20
5000	2	2016/04/18
7000	1	2016/04/18
8000	3	2016/04/19

- This example would return all rows from the *customers* and *orders* tables where there is a matching *customer_id* value in both the *customers* and *orders* tables.
- The rows where *customer_id* is equal to 6000 and 9000 in the *customers* table would be omitted, since they do not exist in both tables. The row where the *order_id* is 5 from the *orders* table would be omitted, since the *customer_id* of NULL does not exist in the *customers* table.

Old Syntax

As a final note, it is worth mentioning that the INNER JOIN example above could be rewritten using the older implicit syntax as follows (but we still recommend using the INNER JOIN keyword syntax):

```
SELECT customers.customer_id, orders.order_id, orders.order_date
FROM customers, orders
WHERE customers.customer_id = orders.customer_id
ORDER BY customers.customer_id;
```

Outer joins

SQL LEFT OUTER JOIN

Another type of join is called a LEFT OUTER JOIN. This type of join returns all rows from the LEFT-hand table specified in the ON condition and **only** those rows from the other table where the joined fields are equal (join condition is met).

Syntax

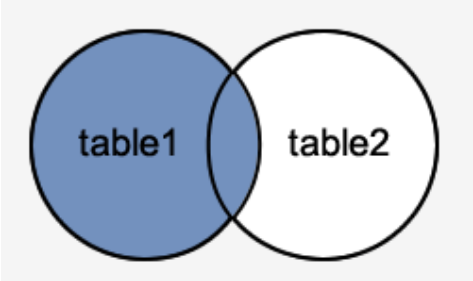
The syntax for the LEFT OUTER JOIN in SQL is:

```
SELECT columns
FROM table1
LEFT [OUTER] JOIN table2
ON table1.column = table2.column;
```

In some databases, the OUTER keyword is omitted and written simply as LEFT JOIN.

Visual Illustration

In this visual diagram, the SQL LEFT OUTER JOIN returns the shaded area:



The SQL LEFT OUTER JOIN would return the all records from *table1* and only those records from *table2* that intersect with *table1*.

Example

Now let's look at an example that shows how to use the LEFT OUTER JOIN in a SELECT statement.

Using the same *customers* table as the previous example:

customer_id	last_name	first_name	favorite_website
4000	Jackson	Joe	techonthenet.com
5000	Smith	Jane	digminecraft.com
6000	Ferguson	Samantha	bigactivities.com
7000	Reynolds	Allen	checkyourmath.com
8000	Anderson	Paige	NULL
9000	Johnson	Derek	techonthenet.com

And the *orders* table with the following data:

order_id	customer_id	order_date
1	7000	2016/04/18
2	5000	2016/04/18

order_id	customer_id	order_date
3	8000	2016/04/19
4	4000	2016/04/20
5	NULL	2016/05/01

Enter the following SQL statement:

```
SELECT customers.customer_id, orders.order_id, orders.order_date
FROM customers
LEFT JOIN orders
ON customers.customer_id = orders.customer_id
ORDER BY customers.customer_id;
```

There will be 6 records selected. These are the results that you should see:

customer_id	order_id	order_date
4000	4	2016/04/20
5000	2	2016/04/18
6000	NULL	NULL
7000	1	2016/04/18
8000	3	2016/04/19
9000	NULL	NULL

This LEFT OUTER JOIN example would return all rows from the *customers* table and only those rows from the *orders* table where the joined fields are equal.

If a *customer_id* value in the *customers* table does not exist in the *orders* table, all fields in the *orders* table will display as NULL in the result set. As you can see, the rows where *customer_id* is 6000 and 9000 would be included with a LEFT OUTER JOIN but the *order_id* and *order_date* fields display NULL.

SQL RIGHT OUTER JOIN

Another type of join is called a SQL RIGHT OUTER JOIN. This type of join returns all rows from the RIGHT-hand table specified in the ON condition and **only** those rows from the other table where the joined fields are equal (join condition is met).

Syntax

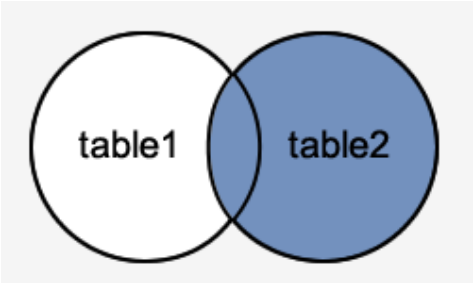
The syntax for the RIGHT OUTER JOIN in SQL is:

```
SELECT columns
FROM table1
RIGHT [OUTER] JOIN table2
ON table1.column = table2.column;
```

In some databases, the OUTER keyword is omitted and written simply as RIGHT JOIN.

Visual Illustration

In this visual diagram, the SQL RIGHT OUTER JOIN returns the shaded area:



The SQL RIGHT OUTER JOIN would return the all records from *table2* and only those records from *table1* that intersect with *table2*.

Example

Now let's look at an example that shows how to use the RIGHT OUTER JOIN in a SELECT statement.

Using the same *customers* table as the previous example:

customer_id	last_name	first_name	favorite_website
4000	Jackson	Joe	techonthenet.com
5000	Smith	Jane	digminecraft.com
6000	Ferguson	Samantha	bigactivities.com
7000	Reynolds	Allen	checkyourmath.com
8000	Anderson	Paige	NULL
9000	Johnson	Derek	techonthenet.com

And the *orders* table with the following data:

order_id	customer_id	order_date
1	7000	2016/04/18
2	5000	2016/04/18
3	8000	2016/04/19
4	4000	2016/04/20
5	NULL	2016/05/01

Enter the following SQL statement:

```
SELECT customers.customer_id, orders.order_id, orders.order_date
FROM customers
RIGHT JOIN orders
ON customers.customer_id = orders.customer_id
ORDER BY customers.customer_id;
```

There will be 5 records selected. These are the results that you should see:

customer_id	order_id	order_date
NULL	5	2016/05/01
4000	4	2016/04/20
5000	2	2016/04/18
7000	1	2016/04/18
8000	3	2016/04/19

This RIGHT OUTER JOIN example would return all rows from the *orders* table and only those rows from the *customers* table where the joined fields are equal.

If a *customer_id* value in the *orders* table does not exist in the *customers* table, all fields in the *customers* table will display as NULL in the result set. As you can see, the row where *order_id* is 5 would be included with a RIGHT OUTER JOIN but the *customer_id* field displays NULL.

SQL FULL OUTER JOIN

Another type of join is called a SQL FULL OUTER JOIN. This type of join returns all rows from the LEFT-hand table and RIGHT-hand table with NULL values in place where the join condition is not met.

Syntax

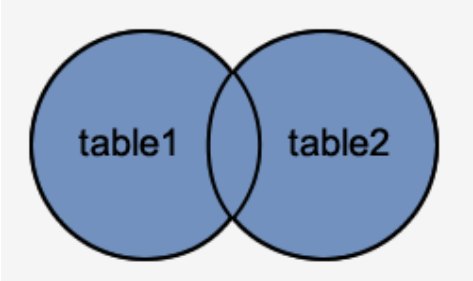
The syntax for the SQL **FULL OUTER JOIN** is:

```
SELECT columns
FROM table1
FULL [OUTER] JOIN table2
ON table1.column = table2.column;
```

In some databases, the OUTER keyword is omitted and written simply as FULL JOIN.

Visual Illustration

In this visual diagram, the SQL FULL OUTER JOIN returns the shaded area:



The SQL FULL OUTER JOIN would return the all records from both *table1* and *table2*.

Example

Let's look at an example that shows how to use the FULL OUTER JOIN in a SELECT statement.

Using the same *customers* table as the previous example:

customer_id	last_name	first_name	favorite_website
4000	Jackson	Joe	techonthenet.com
5000	Smith	Jane	digminecraft.com
6000	Ferguson	Samantha	bigactivities.com
7000	Reynolds	Allen	checkyourmath.com
8000	Anderson	Paige	NULL
9000	Johnson	Derek	techonthenet.com

- And the *orders* table with the following data:

order_id	customer_id	order_date
1	7000	2016/04/18
2	5000	2016/04/18
3	8000	2016/04/19

order_id	customer_id	order_date
4	4000	2016/04/20
5	NULL	2016/05/01

Enter the following SQL statement:

```
SELECT customers.customer_id, orders.order_id, orders.order_date
FROM customers
FULL OUTER JOIN orders
ON customers.customer_id = orders.customer_id
ORDER BY customers.customer_id;
```

There will be 7 records selected. These are the results that you should see:

customer_id	order_id	order_date
NULL	5	2016/05/01
4000	4	2016/04/20
5000	2	2016/04/18
6000	NULL	NULL
7000	1	2016/04/18
8000	3	2016/04/19
9000	NULL	NULL

- This FULL OUTER JOIN example would return all rows from the *orders* table and all rows from the *customers* table. Whenever the joined condition is not met, a NULL value would be extended to those fields in the result set. This means that if a *customer_id* value in the *customers* table does not exist in the *orders* table, all fields in the *orders* table will display as NULL in the result set. Also, if a *customer_id* value in the *orders* table does not exist in the *customers* table, all fields in the *customers* table will display as NULL in the result set.

- As you can see, the rows where the *customer_id* is 6000 and 9000 would be included but the *order_id* and *order_date* fields for those records contains a NULL value. The row where the *order_id* is 5 would be also included but the *customer_id* field for that record has a NULL value.

SQL CROSS JOIN

The SQL CROSS JOIN produces a result set which is the number of rows in the first table multiplied by the number of rows in the second table if no WHERE clause is used along with CROSS JOIN.This kind of result is called as Cartesian Product.

Syntax:

```
SELECT *
FROM table1
CROSS JOIN table2;
```

Example

```
SELECT foods.item_name,foods.item_unit,
company.company_name,company.company_city
FROM foods CROSS JOIN company;
```

SQL - UNIONS CLAUSE

The SQL UNION clause/operator is used to combine the results of two or more SELECT statements without returning any duplicate rows.

To use this UNION clause, each SELECT statement must have

- The same number of columns selected
- The same number of column expressions
- The same data type and
- Have them in the same order

Syntax

The basic syntax of a **UNION** clause is as follows –

```
SELECT column1 [, column2 ]
FROM table1 [, table2 ]
[WHERE condition]

UNION

SELECT column1 [, column2 ]
FROM table1 [, table2 ]
[WHERE condition]
```

```
mysql> select * from master;
+----------+------+----+----------+
| name     | age  | id | subject  |
+----------+------+----+----------+
| rohan    |   20 |  1 | maths    |
| sindhu   |   19 |  2 | social   |
| sinchana |   19 |  3 | science  |
| abhi     |   20 |  4 | computer |
| tej      |   20 |  5 | maths    |
| ankitha  |   20 |  6 | arts     |
+----------+------+----+----------+
6 rows in set (0.13 sec)
```

```
mysql> select * from detail;
+---------+------+
| subject | S_id |
+---------+------+
| maths   |    1 |
| maths   |    5 |
| maths   |    5 |
| physics |    3 |
+---------+------+
```

Example :
mysql> select subject from master union select subject from detail;

Output:

```
+----------+
| subject  |
+----------+
| maths    |
| social   |
| science  |
| computer |
| arts     |
| physics  |
+----------+
```

INTERSECT Clause

The SQL **INTERSECT** clause/operator is used to combine two SELECT statements, but returns rows only from the first SELECT statement that are identical to a row in the second SELECT statement. This means INTERSECT returns only common rows returned by the two SELECT statements.

syntax

```
SELECT column1 [, column2 ]
FROM table1 [, table2 ]
```

```
[WHERE condition]

INTERSECT

SELECT column1 [, column2 ]
FROM table1 [, table2 ]
[WHERE condition]
```

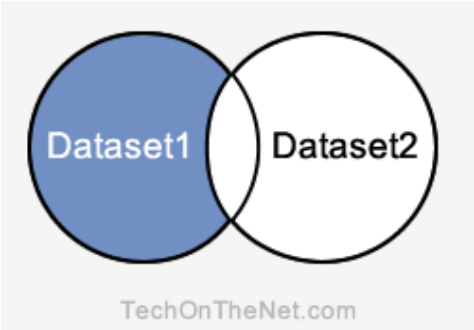
Example: select subject from master intersect select subject from detail;

```
+---------+
| subject |
+---------+
| maths   |
+---------+
```

MINUS Clause

The SQL MINUS operator is used to return all rows in the first SELECT statement that are not returned by the second SELECT statement. Each SELECT statement will define a dataset. The MINUS operator will retrieve all records from the first dataset and then remove from the results all records from the second dataset.

Minus Query



Syntax:

```
SELECT column1 [, column2 ]
FROM table1 [, table2 ]
[WHERE condition]

MINUS

SELECT column1 [, column2 ]
FROM table1 [, table2 ]
[WHERE condition]
```

Example:

select subject from master minus select subject from detail;

Output

